

Customer Purchase Behavior Analysis and Prediction using Hadoop and Apache Spark

Team Members:

Pradeep Rajkumar R

J Nithiasri

DATASET DESCRIPTION

The dataset used in this project is the **E-Commerce Behavior Data from a Multi-Category Online Store**, publicly available through Kaggle and collected by the Open CDP project. The study utilizes the November 2019 dataset, which contains approximately **67.5 million customer interaction records** and occupies nearly **9 GB** of storage.

The dataset captures customer activities on an online shopping platform, including **product views, cart additions, cart removals, and purchase transactions**. It consists of **9 attributes** as listed below:

Attribute	Type
event_time	Temporal (Timestamp)
event_type	Categorical
product_id	Numerical (Identifier)
category_id	Numerical (Identifier)
category_code	Categorical
brand	Categorical
price	Numerical
user_id	Numerical (Identifier)
user_session	Categorical

The dataset contains a mixture of **temporal, categorical, and numerical features**, making it suitable for customer behavior analysis, purchase pattern identification, and large-scale e-commerce analytics. Due to its high volume and millions of user interactions, **Hadoop HDFS** is used for distributed storage, and **Apache Spark** is used for scalable data preprocessing and analytical operations.

Dataset Loading and Environment Setup

1. Starting Hadoop Distributed File System (HDFS):

```
hadoop@pradeep-MacBookAir:~/hadoop$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [pradeep-MacBookAir]
```

2. Verifying Dataset Availability in Local Storage:

```
hadoop@pradeep-MacBookAir:~/hadoop$ ls -lh /home/pradeep/Downloads/customer_behavior.csv
-rw-rw-r-- 1 pradeep pradeep 8.4G Dec  9 2019 /home/pradeep/Downloads/customer_behavior.csv
```

3. Uploading Dataset from Local Storage to HDFS:

```
hadoop@pradeep-MacBookAir:~/hadoop$ bin/hdfs dfs -put /home/pradeep/Downloads/customer_behavior.csv /bigdata_project/
```

4. Verifying Dataset Storage in HDFS:

```
hadoop@pradeep-MacBookAir:~/hadoop$ bin/hdfs dfs -ls /bigdata_project
Found 1 items
-rw-r--r-- 1 hadoop supergroup 9006762395 2026-06-05 13:18 /bigdata_project/customer_behavior.csv
```

5. Configuring Apache Spark Environment Variables:

```
hadoop@pradeep-MacBookAir:~/hadoop$ export SPARK_HOME=$HOME/spark
hadoop@pradeep-MacBookAir:~/hadoop$ export PATH=$PATH:$SPARK_HOME/bin
```

6. Verifying Apache Spark Installation:

```
hadoop@pradeep-MacBookAir:~/hadoop$ spark-shell --version
26/06/05 13:21:39 WARN Utils: Your hostname, pradeep-MacBookAir resolves to a loopback address: 127.0.1.1; using 192.168.1.12 instead (on interface wlp3s0)
26/06/05 13:21:39 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Welcome to

  ____  __
 / ___/ /  \
/ /   / _  \
/ /___/ ___ \
\____/_/  \_\
        version 3.5.1

Using Scala version 2.12.18, OpenJDK 64-Bit Server VM, 17.0.18
Branch HEAD
Compiled by user heartsavior on 2024-02-15T11:24:58Z
Revision fd86f85e181fc2dc0f50a096855acf83a6cc5d9c
Url https://github.com/apache/spark
Type --help for more information.
```

7. Launching Apache Spark Shell:

```
hadoop@pradeep-MacBookAir:~/hadoop$ spark-shell
26/06/05 13:46:48 WARN Utils: Your hostname, pradeep-MacBookAir resolves to a loopback address: 127.0.1.1; using 192.168.1.12 instead (on interface wlp3s0)
26/06/05 13:46:48 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/06/05 13:46:57 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark context Web UI available at http://pradeep-MacBookAir.domain.name:4040
Spark context available as 'sc' (master = local[*], app id = local-1780647418743).
Spark session available as 'spark'.
Welcome to

  ____  __
 / ___/ /  \
/ /   / _  \
/ /___/ ___ \
\____/_/  \_\
        version 3.5.1

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 17.0.18)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

8. Loading Dataset from HDFS into Spark DataFrame:

```
scala> val df=spark.read.option("header","true").csv("hdfs://localhost:9000/bigdata_project/customer_behavior.csv")
df: org.apache.spark.sql.DataFrame = [event_time: string, event_type: string ... 7 more fields]
```

9. Inspecting Dataset Schema:

```
scala> df.printSchema()
root
|-- event_time: string (nullable = true)
|-- event_type: string (nullable = true)
|-- product_id: string (nullable = true)
|-- category_id: string (nullable = true)
|-- category_code: string (nullable = true)
|-- brand: string (nullable = true)
|-- price: string (nullable = true)
|-- user_id: string (nullable = true)
|-- user_session: string (nullable = true)
```

10. Displaying Sample Records from Dataset:

```
scala> df.show(5,false)
+-----+-----+-----+-----+-----+-----+-----+-----+
|event_time|event_type|product_id|category_id|category_code|brand|price|user_id|user_session|
+-----+-----+-----+-----+-----+-----+-----+-----+
|2019-11-01 00:00:00 UTC|view|1003461|2053013555631882655|electronics.smartphone|xiaomi|489.07|520088904|4d3b30da-a5e4-49df-b1a8-ba5943f1dd33|
|2019-11-01 00:00:00 UTC|view|5000088|2053013566100866035|appliances.sewing_machine|janome|293.65|530496790|8e5f4f83-366c-4f70-860e-ca7417414283|
|2019-11-01 00:00:01 UTC|view|17302664|2053013553853497655|NULL|creed|28.31|561587266|755422e7-9040-477b-9bd2-6a6e8fd97387|
|2019-11-01 00:00:01 UTC|view|3601530|2053013563810775923|appliances.kitchen.washer|lg|712.87|518085591|3bfb58cd-7892-48cc-8020-2f17e6de6e7f|
|2019-11-01 00:00:01 UTC|view|1004775|2053013555631882655|electronics.smartphone|xiaomi|183.27|558856683|313628f1-68b8-460d-84f6-cec7a8796ef2|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

11. Determining Total Number of Records (67.5 million records):

```
scala> df.count()
res1: Long = 67501979
```

12. Determining Total Number of Attributes:

```
scala> println(df.columns.length)
9
```

Data Preprocessing

13. Data Type Conversion and Null Value Standardization:

Converted important attributes to appropriate data types and transformed "NULL" string values into actual null values for accurate processing.

```
scala> import org.apache.spark.sql.functions._
import org.apache.spark.sql.functions._
```

```
scala> val df2=df.withColumn("event_time",to_timestamp(col("event_time"),"yyyy-MM-dd HH:mm:ss z")).w
ithColumn("product_id",col("product_id").cast("long")).withColumn("category_id",col("category_id").c
ast("long")).withColumn("price",col("price").cast("double")).withColumn("user_id",col("user_id").cas
t("long")).withColumn("category_code",when(col("category_code")==="NULL",null).otherwise(col("catego
ry_code"))).withColumn("brand",when(col("brand")==="NULL",null).otherwise(col("brand")))
df2: org.apache.spark.sql.DataFrame = [event_time: timestamp, event_type: string ... 7 more fields]
```

14. Verification of Updated Schema:

```
scala> df2.printSchema()
root
|-- event_time: timestamp (nullable = true)
|-- event_type: string (nullable = true)
|-- product_id: long (nullable = true)
|-- category_id: long (nullable = true)
|-- category_code: string (nullable = true)
|-- brand: string (nullable = true)
|-- price: double (nullable = true)
|-- user_id: long (nullable = true)
|-- user_session: string (nullable = true)
```

15. Missing Value Identification:

Identify missing values in all attributes.

```
scala> df2.select(df2.columns.map(c=>sum(col(c).isNull.cast("int")).alias(c)):_*).show(false)
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|event_time|event_type|product_id|category_id|category_code|brand  |price|user_id|user_session|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|0         |0         |0         |0         |21898171    |9218235|0    |0      |10         |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

16. Removal of Records with Missing Brand and Category Information:

```
scala> val cleanDF=df2.na.drop(Seq("brand","category_code"))
cleanDF: org.apache.spark.sql.DataFrame = [event_time: timestamp, event_type: string ... 7 more fields]
```

17. Record Count After Null Value Removal (42.1 million high-quality records remained for analysis):

```
scala> cleanDF.count()
res12: Long = 42089570
```

Exploratory Data Analysis

18. Event Type Distribution:

```
scala> cleanDF.groupBy("event_type").count().show(false)
+-----+-----+
|event_type|count  |
+-----+-----+
|purchase  |659256 |
|view      |39315226|
|cart      |2115088 |
+-----+-----+
```

Purpose: Analyze the distribution of customer activities such as product views, cart additions, purchases, and cart removals.

19. Top 10 Brands by Customer Interactions:

```
scala> cleanDF.groupBy("brand").count().orderBy(desc("count")).show(10, false)
+-----+-----+
|brand  |count  |
+-----+-----+
|samsung|7733328|
|apple  |6213902|
|xiaomi  |4138112|
|huawei  |1384154|
|lg      |1024251|
|oppo    |811698 |
|respect |732666 |
|lenovo  |727279 |
|acer    |698910 |
|bosch   |605523 |
+-----+-----+
only showing top 10 rows
```

Purpose: Identify the most frequently interacted brands.

20. Top 10 Product Categories:

```
scala> cleanDF.groupBy("category_code").count().orderBy(desc("count")).show(10,false)
+-----+-----+
|category_code|count|
+-----+-----+
|electronics.smartphone|16353582|
|electronics.video.tv|2195119|
|computers.notebook|2164657|
|electronics.clocks|1811325|
|electronics.audio.headphone|1803895|
|apparel.shoes|1587667|
|appliances.environment.vacuum|1510004|
|appliances.kitchen.washer|1389808|
|appliances.kitchen.refrigerators|1149533|
|computers.desktop|647867|
+-----+-----+
only showing top 10 rows
```

Purpose: Determine the most popular product categories.

21. Distribution of Purchase Events:

```
scala> cleanDF.filter(col("event_type")==="purchase").count()
res19: Long = 659256
```

Purpose: Calculate the total number of purchase transactions.

22. Top Purchased Brands:

```
scala> cleanDF.filter(col("event_type")==="purchase").groupBy("brand").count().orderBy(desc("count")).show(10,false)
+-----+-----+
|brand|count|
+-----+-----+
|samsung|198670|
|apple|165681|
|xiaomi|57909|
|huawei|23466|
|oppo|15080|
|lg|11828|
|artel|7269|
|lenovo|6546|
|acer|6402|
|bosch|5718|
+-----+-----+
only showing top 10 rows
```

23. Revenue Analysis:

```
scala> cleanDF.filter(col("event_type")==="purchase").agg(format_number(sum("price"),2)
.alias("Total_Revenue")).show(false)
+-----+
|Total_Revenue |
+-----+
|241,560,392.08|
+-----+
```

Purpose: Calculate total revenue generated from purchase transactions.

24. Most Active Users:

```
scala> cleanDF.groupBy("user_id").count().orderBy(desc("count")).show(10,false)
+-----+-----+
|user_id |count|
+-----+-----+
|569335945|12644|
|568778435|8829 |
|512365995|5982 |
|512475445|4799 |
|568804062|4179 |
|512388419|3646 |
|567475167|3356 |
|568781398|2939 |
|514649263|2865 |
|568805468|2809 |
+-----+-----+
only showing top 10 rows
```

Purpose: Identify users with the highest interaction frequency.